



Payment Service

API Documentation

Technology Stack: Kotlin, Ktor, Koin, Restate, TigerBeetle, PostgreSQL, Exposed, HikariCP

V 1.0.0

1. Overview

Purpose

The Payment Service API provides endpoints for managing:

- Customers
- Merchants
- Systems
- Accounts
- Payments
- Account transfers (ledger transfers using TigerBeetle)

The API separates the management of application entities from the processing of financial transactions.

Customers, merchants, systems, accounts, and payment records are managed as application entities. Financial movement between accounts is processed through the payment workflow and recorded in the financial ledger.

Base URL

<http://localhost:8080>

Content Type

application/json

Requests that contain a body must use JSON unless otherwise specified.

Responses are returned as JSON.

The application also exposes generated API documentation.

Swagger UI

<http://localhost:8080/swagger>

OpenAPI

<http://localhost:8080/openapi>

2. General API Conventions

Request Format

Requests that create or update resources are submitted as JSON request bodies.

Path parameters are used to identify specific resources, such as customers, merchants, systems, accounts, and payments.

Query parameters are used for optional filtering criteria (to-from date ranges).

Response Format

The API uses a consistent result structure for application responses.

Successful Response

A successful operation returns a **SUCCESS** result code together with response data.

```
{
  "resultCode": "SUCCESS",
  "data": {}
}
```

Failed Response

A failed operation returns a **FAILED** result code together with one or more errors.

```
{
  "resultCode": "FAILED",
  "errors": [
    {
      "code": "ERROR_CODE",
      "description": "Error description."
    }
  ]
}
```

The **code** field identifies the specific failure condition.

The **description** field provides a human-readable explanation.

3. Result Codes

Result Code	Description
SUCCESS	The request completed successfully and the requested operation was performed.
FAILED	The request could not be completed because of validation, business, or processing errors.

4. Customers API

The Customers API manages customer records used by the payment system.

A customer can own many accounts through the CUSTOMER owner category.

4.1 Create Customer

Endpoint

POST /customers

Request

```
{
  "msgId": "customer-create-001",
  "title": "Mr",
  "firstName": "John",
  "lastName": "Smith",
  "identityNumber": "9001015009087",
  "passportNumber": null,
  "country": "ZA",
  "mobileNumber": "+27123456789",
  "email": "john.smith@example.com",
  "physicalAddress": "1 Main Street, Johannesburg"
}
```

Request Fields

Field	Required	Description
msgId	Yes	Message identifier for the request.
title	Yes	Customer title.
firstName	Yes	Customer first name.
lastName	Yes	Customer last name.
identityNumber	Yes	Customer identity number.
passportNumber	No	Optional passport number.
country	Yes	Customer country.
mobileNumber	Yes	Customer mobile number.
email	Yes	Customer email address.
physicalAddress	Yes	Customer physical address.

Success Response

Returns HTTP 201 Created.

The response contains the created customer id.

4.2 Get All Customers

Endpoint

GET /customers

Returns all customer records.

Success Response

Returns HTTP 200 OK.

4.3 Get Customer

Endpoint

GET /customers/{customerId}

Retrieves a specific customer using its identifier.

customerId The UUID identifier of the customer.

Path Parameters

customerId The UUID identifier of the customer.

Success Response

Returns HTTP 200 OK.

Returns the requested customer when it exists.

Errors

Error Code	Description
CUSTOMER_NOT_FOUND	The specified customer does not exist.
INVALID_CUSTOMER_ID	The supplied customer identifier is invalid.

4.4 Update Customer

Endpoint

PUT /customers/{customerId}

Updates the supplied customer fields.

Request

All update fields other than msgId are optional.

```
{
  "msgId": "customer-update-001",
  "status": "ACTIVE",
  "title": "Mr",
  "firstName": "John",
  "lastName": "Smith",
  "identityNumber": "9001015009087",
  "passportNumber": null,
  "country": "ZA",
  "mobileNumber": "+27123456789",
  "email": "john.smith@example.com",
  "physicalAddress": "2 Main Street, Johannesburg"
}
```

Success Response

Returns HTTP 200 OK.

5. Merchants API

The Merchants API manages merchant records used by the payment system.

A merchant can own many accounts through the MERCHANT owner category.

5.1 Create Merchant

Endpoint

POST /merchants

Request

```
{
  "msgId": "merchant-create-001",
  "type": "AIRTIME",
  "name": "Example Merchant",
  "registrationNumber": "REG-001",
  "country": "ZA",
  "mobileNumber": "+27123456789",
  "email": "merchant@example.com",
  "physicalAddress": "1 Merchant Street, Johannesburg"
}
```

Request Fields

Field	Required	Description
msgId	Yes	Message identifier for the request.
type	Yes	Merchant type.
name	Yes	Merchant name.
registrationNumber	Yes	Merchant registration number.
country	Yes	Merchant country.
mobileNumber	Yes	Merchant contact number.
email	Yes	Merchant email address.
physicalAddress	Yes	Merchant physical address.

Success Response

Returns HTTP 201 Created.

5.2 Get All Merchants

Endpoint

GET /merchants

Returns all merchant records.

Success Response

Returns HTTP 200 OK.

5.3 Get Merchant

Endpoint

GET /merchants/{merchantId}

Retrieves a specific merchant using its identifier.

Path Parameters

merchantId The UUID identifier of the merchant.

Success Response

Returns HTTP 200 OK.

Errors

Error Code	Description
MERCHANT_NOT_FOUND	The specified merchant does not exist.
INVALID_MERCHANT_ID	The supplied merchant identifier is invalid.

5.4 Update Merchant

Endpoint

PUT /merchants/{merchantId}

Updates the supplied merchant fields.

Request

```
{  
  "msgId": "merchant-update-001",  
  "status": "ACTIVE",  
  "type": "AIRTIME",  
  "name": "Updated Merchant",  
  "registrationNumber": "REG-001",  
  "country": "ZA",  
  "mobileNumber": "+27123456789",  
  "email": "merchant@example.com",  
  "physicalAddress": "2 Merchant Street, Johannesburg"  
}
```

All fields other than msgId are optional.

Success Response

Returns HTTP 200 OK.

6. Systems API

The Systems API manages internal system entities (used for settlements, fees, reservation etc).

System entities can own many system-managed accounts through the SYSTEM owner category.

Examples include internal accounts for settlement, fees, reservation etc.

6.1 Create System

Endpoint

POST /systems

Request

```
{  
  "msgId": "system-create-001",  
  "name": "Payment System",  
  "description": "Internal payment processing system"  
}
```

Request Fields

Field	Required	Description
msgId	Yes	Message identifier for the request.
name	Yes	System name.
description	Yes	Description of the system.

Success Response

Returns HTTP 201 Created.

6.2 Get All Systems

Endpoint

GET /systems

Returns all system entities.

Success Response

Returns HTTP 200 OK.

6.3 Get System

Endpoint

GET /systems/{systemId}

Retrieves a specific system using its identifier.

Path Parameters

systemId The UUID identifier of the system.

Success Response

Returns HTTP 200 OK.

Errors

Error Code	Description
SYSTEM_NOT_FOUND	The specified system does not exist.
INVALID_SYSTEM_ID	The supplied system identifier is invalid.

6.4 Update System

Endpoint

PUT /systems/{systemId}

Updates the supplied system fields.

Request

```
{  
  "msgId": "system-update-001",  
  "name": "Updated Payment System",  
  "description": "Updated system description"  
}
```

All fields other than msgId are optional.

Success Response

Returns HTTP 200 OK.

7. Accounts API

The Accounts API manages application account metadata.

Financial balances and transfers are maintained separately in the financial ledger.

Accounts belong to one of the supported owner categories:

- CUSTOMER
- MERCHANT
- SYSTEM

7.1 Create Account

Endpoint

POST /accounts

Request

```
{  
  "msgId": "account-create-001",  
  "ownerId": "11111111-1111-1111-1111-111111111111",  
  "ownerCategory": "CUSTOMER",  
  "ledger": "PAYMENTS",  
  "accountType": "TRANSACTIONAL",  
  "description": "Customer transactional account"  
}
```

Request Fields

Field	Required	Description
msgId	Yes	Message identifier for the request.
ownerId	Yes	UUID of the account owner.
ownerCategory	Yes	Category of the account owner.
ledger	Yes	Ledger to which the account belongs.
accountType	Yes	Classification of the account.
description	Yes	Account description.

Success Response

Returns HTTP 201 Created.

7.2 Get All Accounts

Endpoint

GET /accounts

Returns all application accounts.

Success Response

Returns HTTP 200 OK.

7.3 Get Account

Endpoint

GET /accounts/{accountNumber}

Retrieves an account using its account number.

Path Parameters

accountNumber	The account number to retrieve.
---------------	---------------------------------

Success Response

Returns HTTP 200 OK.

Errors

Error Code	Description
ACCOUNT_NOT_FOUND	The specified account does not exist.

7.4 Update Account

Endpoint

PUT /accounts/{accountNumber}

Updates the supplied account fields.

Request

```
{  
  "msgId": "account-update-001",  
  "status": "ACTIVE",  
  "accountType": "TRANSACTIONAL",  
  "description": "Updated account description"  
}
```

Request Fields

Field	Required	Description
msgId	Yes	Message identifier for the request.
status	No	Updated account status.
accountType	No	Updated account type.
description	No	Updated account description.

At least one updateable field should be supplied where required by the application.

Success Response

Returns HTTP 200 OK.

Errors

Error Code	Description
ACCOUNT_NOT_FOUND	The specified account does not exist.
INVALID_ACCOUNT_STATUS	The supplied account status is invalid.
INVALID_ACCOUNT_TYPE	The supplied account type is invalid.

7.5 Get Account Transfers

Endpoint

GET /accounts/{accountNumber}/transfers

Retrieves financial transfer history for the specified account from the ledger.

Path Parameters

accountNumber The account number whose transfers are requested.

Success Response

Returns HTTP 200 OK.

The response represents ledger transfer information rather than application payment history.

8. Payments API

The Payments API processes financial payments and retrieves application payment records. Payment processing uses the payment workflow and financial ledger.

8.1 Create Payment

Endpoint

POST /payments

Request

```
{
  "msgId": "payment-001",
  "debtorAccountNumber": "1001",
  "creditorAccountNumber": "1002",
  "amount": 10000,
  "asset": "ZAR",
  "assetType": "FIAT",
  "type": "MOBILE_TOP_UP"
}
```

Request Fields

Field	Required	Description
msgId	Yes	Unique message identifier used as the payment workflow identity.
debtorAccountNumber	Yes	Account from which funds are debited.
creditorAccountNumber	Yes	Account to which funds are credited.
amount	Yes	Payment amount represented in integer minor units.
asset	Yes	Currency or financial asset.
assetType	Yes	Classification of the asset.
type	Yes	Payment type.

Amount Format

Payment amounts are represented using whole integers.

For example, for a two-decimal currency:

10000 = 100.00

Floating-point values are not used for payment amounts.

Processing

A payment request performs the following high-level processing:

1. Validates the payment amount.
2. Validates that debtor and creditor accounts are different.
3. Validates that both accounts exist.
4. Validates that both participating accounts are active.
5. Retrieves the applicable fee configuration.
6. Calculates the payment fee.
7. Creates the application payment record.
8. Creates the payment transfer and optional fee transfer in the financial ledger.
9. Updates the payment status according to the workflow outcome.

When a fee applies, the payment transfer and fee transfer are processed as a linked ledger batch.

Success Response

Returns HTTP 201 Created.

```
{
  "resultCode": "SUCCESS",
  "data": {
    "paymentReference": "11111111-1111-1111-1111-111111111111",
    "feeReference": "22222222-2222-2222-2222-222222222222"
  }
}
```

feeReference may be null when no fee transfer is created (this would occur when the fee amount is less than a single whole integer [1]).

Errors

Error Code	Description
INVALID_TRANSACTION_AMOUNT	The payment amount must be greater than zero.
INVALID_ACCOUNTS	The debtor and creditor accounts cannot be the same.
DEBTOR_ACCOUNT_NOT_FOUND	The debtor account does not exist.

Error Code	Description
CREDITOR_ACCOUNT_NOT_FOUND	The creditor account does not exist.
DEBTOR_ACCOUNT_NOT_ACTIVE	The debtor account is not active.
CREDITOR_ACCOUNT_NOT_ACTIVE	The creditor account is not active.
INSUFFICIENT_FUNDS	The debtor account does not have sufficient funds for the required ledger transfer.
FEE_CONFIGURATION_NOT_FOUND	No applicable fee configuration was found for the payment type and asset.
DUPLICATE_MESSAGE_ID	The supplied message ID has already been used to invoke the payment workflow.
INVALID_ASSET	The supplied asset is invalid.
INVALID_ASSET_TYPE	The supplied asset type is invalid.
INVALID_PAYMENT_TYPE	The supplied payment type is invalid.

8.2 Get All Payments

Endpoint

GET /payments

Returns application payment records.

Optional Query Parameters

Parameter Description

from Optional lower bound for the payment creation date range.

to Optional upper bound for the payment creation date range.

Examples

GET /payments

GET /payments?from=2026-08-25T00:00:00

GET /payments?to=2026-08-25T23:59:59

GET /payments?from=2026-08-25T00:00:00&to=2026-08-25T23:59:59

Payment filtering uses LocalDateTime values.

When supplied:

createdDate >= from

createdDate <= to

Results are ordered by creation date, newest first.

Errors

Error Code	Description
INVALID_FROM_DATE	The supplied from date is invalid.
INVALID_TO_DATE	The supplied to date is invalid.
INVALID_DATE_RANGE	The from date occurs after the to date.

8.3 Get Payment

Endpoint

GET /payments/{paymentReference}

Retrieves a payment using its payment reference.

Path Parameters

paymentReference UUID payment reference.

Success Response

Returns HTTP 200 OK.

Errors

Error Code	Description
INVALID_PAYMENT_REFERENCE	The supplied payment reference is invalid.
PAYMENT_NOT_FOUND	The requested payment does not exist.

9. Enum Reference

Owner Category

The owner category identifies the type of entity that owns an account.

Value	Description
CUSTOMER	The account belongs to a customer.
MERCHANT	The account belongs to a merchant.
SYSTEM	The account belongs to an internal system entity.

Owner Status

The owner status identifies the operational state of a customer or merchant.

Value	Description
ACTIVE	The owner is active.
INACTIVE	The owner is inactive.
SUSPENDED	The owner is suspended.

Account Status

The account status determines whether an account is allowed to perform actions that require an active account, such as payments.

Value	Description
ACTIVE	The account is active and may participate in supported operations.
SUSPENDED	The account is suspended and is not eligible for operations requiring an active account.
RESTRICTED	The account has restrictions and is not eligible for operations requiring an active account.
DORMANT	The account is dormant and is not eligible for operations requiring an active account.
CLOSED	The account is closed and cannot participate in account operations.

Account Type

The account type identifies the classification of an account.

Value	Description
TRANSACTIONAL	Standard account used for transactional operations.
SETTLEMENT	Account used for settlement operations.
FEE	Account used for fee collection.
RESERVE	Account used for reserve purposes.

Merchant Type

The merchant type identifies the category of merchant.

Value	Description
AIRTIME	Merchant providing airtime-related services.
ELECTRICITY	Merchant providing electricity-related services.
GAMING	Merchant providing gaming-related services.
RETAIL	Merchant providing retail services.

Payment Status

The payment status represents the lifecycle state of an application payment record.

Value	Description
PENDING	The payment has been created and is awaiting completion of processing.
SUCCESS	The payment completed successfully.
FAILED	A payment failure state available in the domain lifecycle.
REJECTED	The payment was rejected due to an expected business or ledger outcome.
REVERSED	A payment reversal state available for extended lifecycle handling.

The normal V1 payment creation workflow primarily transitions newly created payments through:

PENDING → SUCCESS

PENDING → REJECTED

Payment Type

The payment type identifies the category of payment being processed.

Value	Description
MOBILE_TOP_UP	Mobile airtime or top-up payment.
ELECTRICITY_PURCHASE	Electricity purchase payment.
GAMING_VOUCHER	Gaming voucher payment.
RETAIL_TOP_UP	Retail top-up payment.

Payment type is used when determining the applicable fee configuration.

Asset

The asset identifies the currency or financial asset associated with a payment.

Value	Description
USD	United States Dollar
GBP	British Pound Sterling
EUR	Euro
ZAR	South African Rand

Asset Type

The asset type identifies the classification of an asset.

Value	Description
FIAT	Fiat currency asset.

Ledger

The ledger identifies the financial ledger classification used by an account or transfer.

Value	Description
PAYMENTS	Payment ledger.
SETTLEMENT	Settlement ledger.

10. Validation

The API validates request bodies, path parameters, query parameters, and enum values.

Invalid enum values are rejected with a corresponding error code.

Examples include:

- INVALID_OWNER_CATEGORY
- INVALID_OWNER_STATUS
- INVALID_ACCOUNT_STATUS
- INVALID_ACCOUNT_TYPE
- INVALID_MERCHANT_TYPE
- INVALID_ASSET
- INVALID_ASSET_TYPE
- INVALID_PAYMENT_TYPE

Malformed or unsupported request bodies may return:

INVALID_REQUEST

11. Payment Message Identity and Duplicate Prevention

Message ID

For payment processing, msgId identifies the payment workflow.

The workflow identity is derived from the payment message ID.

Conceptually:

msgId

↓

payment-{msgId}

↓

Restate Workflow Identity

Behaviour

- A unique msgId identifies a payment workflow.
- Different msgId values create different workflow identities.
- Reusing the same msgId addresses the same workflow instance.
- Once the run workflow method has been invoked for that workflow instance, it cannot be invoked again as a new payment.
- A duplicate invocation is translated by the API into DUPLICATE_MESSAGE_ID.
- Clients must use a new unique msgId when submitting a new payment.

msgId should therefore be treated as the identifier for a logical payment submission.

12. HTTP Status Codes

The API uses HTTP status codes to communicate the outcome of requests.

Status	Meaning
200 OK	The request was processed successfully.
201 Created	A new resource or payment was successfully created.
400 Bad Request	The request could not be processed because of invalid input, validation failure, or an application-level business failure.
404 Not Found	The requested resource was not found where the endpoint returns an explicit not-found response.
500 Internal Server Error	An unexpected server-side error occurred while processing the request.

13. Error Codes Reference

This section provides a consolidated reference for important API error codes.

Error Code	Description
CUSTOMER_NOT_FOUND	The specified customer does not exist.
MERCHANT_NOT_FOUND	The specified merchant does not exist.
SYSTEM_NOT_FOUND	The specified system does not exist.
ACCOUNT_NOT_FOUND	The specified account does not exist.
PAYMENT_NOT_FOUND	The specified payment does not exist.
INVALID_TRANSACTION_AMOUNT	The payment amount must be greater than zero.
INVALID_ACCOUNTS	The debtor and creditor accounts cannot be the same.
DUPLICATE_MESSAGE_ID	A payment with this message ID has already been processed.
INSUFFICIENT_FUNDS	The debtor account does not have sufficient funds.
DEBTOR_ACCOUNT_NOT_FOUND	The specified debtor account does not exist.
CREDITOR_ACCOUNT_NOT_FOUND	The specified creditor account does not exist.
DEBTOR_ACCOUNT_NOT_ACTIVE	The debtor account is not active.
CREDITOR_ACCOUNT_NOT_ACTIVE	The creditor account is not active.
FEE_CONFIGURATION_NOT_FOUND	No applicable fee configuration was found.
INVALID_FROM_DATE	The supplied from date is invalid.
INVALID_TO_DATE	The supplied to date is invalid.
INVALID_DATE_RANGE	The supplied date range is invalid.
INVALID_PAYMENT_REFERENCE	The supplied payment reference is invalid.
INVALID_CUSTOMER_ID	The supplied customer identifier is invalid.
INVALID_MERCHANT_ID	The supplied merchant identifier is invalid.
INVALID_SYSTEM_ID	The supplied system identifier is invalid.
INVALID_OWNER_CATEGORY	The supplied owner category is invalid.
INVALID_OWNER_STATUS	The supplied owner status is invalid.
INVALID_ACCOUNT_STATUS	The supplied account status is invalid.
INVALID_ACCOUNT_TYPE	The supplied account type is invalid.
INVALID_MERCHANT_TYPE	The supplied merchant type is invalid.
INVALID_ASSET	The supplied asset is invalid.
INVALID_ASSET_TYPE	The supplied asset type is invalid.
INVALID_PAYMENT_TYPE	The supplied payment type is invalid.
INVALID_REQUEST	The request body or request data is invalid.
LEDGER_OPERATION_FAILED	An unexpected ledger operation failed.
UNEXPECTED_ERROR	An unexpected server-side error occurred.

14. Example Requests and Responses

Successful Payment

Request

```
{  
  "msgId": "payment-example-001",  
  "debtorAccountNumber": "1001",  
  "creditorAccountNumber": "1002",  
  "amount": 10000,  
  "asset": "ZAR",  
  "assetType": "FIAT",  
  "type": "MOBILE_TOP_UP"  
}
```

Response

HTTP:

201 Created

Body:

```
{  
  "resultCode": "SUCCESS",  
  "data": {  
    "paymentReference": "11111111-1111-1111-1111-111111111111",  
    "feeReference": "22222222-2222-2222-2222-222222222222"  
  }  
}
```

Failed Payment

Request

```
{  
  "msgId": "payment-example-invalid-001",  
  "debtorAccountNumber": "1001",  
  "creditorAccountNumber": "1001",  
  "amount": 10000,  
  "asset": "ZAR",  
  "assetType": "FIAT",  
  "type": "MOBILE_TOP_UP"  
}
```

Response

HTTP:

400 Bad Request

Body:

```
{  
  "resultCode": "FAILED",  
  "errors": [  
    {  
      "code": "INVALID_ACCOUNTS",  
      "description": "Debtor Account and Creditor Account cannot be the same."  
    }  
  ]  
}
```

Duplicate Payment Message ID

Submitting a payment using a msgId that has already invoked the same payment workflow is rejected.

Response

HTTP:

400 Bad Request

Body:

```
{  
  "resultCode": "FAILED",  
  "errors": [  
    {  
      "code": "DUPLICATE_MESSAGE_ID",  
      "description": "A payment with this message ID has already been processed."  
    }  
  ]  
}
```
